

Documentație Tehnică Completă: Unemployment Data Visualizer

Bazon Bogdan
Meraru Ioan-Lucian

May 20, 2026

Contents

1	Problem Definition	3
2	Use Case Scenario	3
2.1	Fluxul Principal	3
2.2	Gestionarea Erorilor (Ce se întâmplă dacă ceva nu merge bine)	3
3	Diagrams (C4 Model)	3
3.1	Level 1: System Context Diagram	3
3.2	Level 2: Container Diagram	4
3.3	Level 3: Component Diagram (Backend API)	4
3.4	Level 4: Code Diagram (Class Structure Example)	4
4	Backend Documentation	5
4.1	API Reference	5
4.1.1	Endpoint: Fetch Data	5
4.1.2	Endpoint: List Cache	5
4.1.3	Endpoint: Delete Cache	5
4.2	DTOs Structure (Models)	5
4.2.1	UnemploymentDataBasic	5
4.2.2	UnemploymentDataPerAgeRange	6
4.2.3	UnemploymentDataPerEducationLevel	6
4.2.4	UnemploymentDataPerMedium	6
4.3	Caching and Transformation Strategy	6
4.3.1	Mecanismul de Stocare	6
4.3.2	Ciclul de Viață al Datelor (TTL)	6
4.3.3	De ce convertim din CSV în JSON?	7
5	Frontend Implementation	8
5.1	Mecanismul de Export	8
5.1.1	Export CSV (Date Tabelare)	8
5.1.2	Export SVG (Vectorial)	8
5.1.3	Export PDF (Raport Complet)	8
5.2	Harta Interactivă (Leaflet)	8

5.3	Randarea Graficelor (Chart.js)	9
-----	--	---

1 Problem Definition

Acest proiect presupune dezvoltarea unei aplicații web ce afișează datele despre șomajul în România prin anumite criterii (Rata de Șomaj, Mediul de proveniență, Vârstă, Nivel de Educație, etc.) și o anumită dată. Datele sunt preluate din **data.gov.ro** și aplicația are posibilitatea de a compara datele dintr-o anumită perioadă cu alta. Aplicația permite exportul de date prin CSV și PDF plus exportul grafurilor în SVG. De asemenea, aplicația dispune și de un sistem de caching pentru îmbunătățirea preluării datelor.

2 Use Case Scenario

2.1 Fluxul Principal

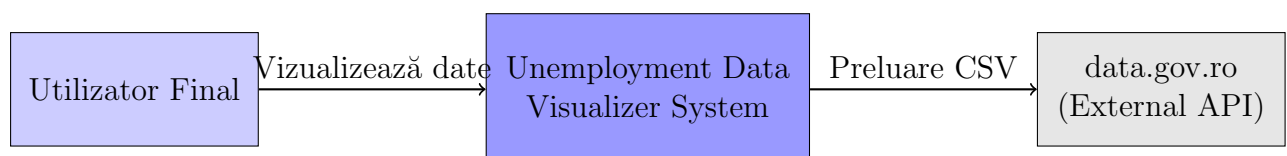
Utilizatorul accesează platforma pentru a analiza evoluția șomajului. Poate filtra datele după lună, județ sau criteriu (vârstă, educație, mediu). Sistemul permite compararea a două perioade diferite pentru a observa tendințele economice.

2.2 Gestionarea Erorilor (Ce se întâmplă dacă ceva nu merge bine)

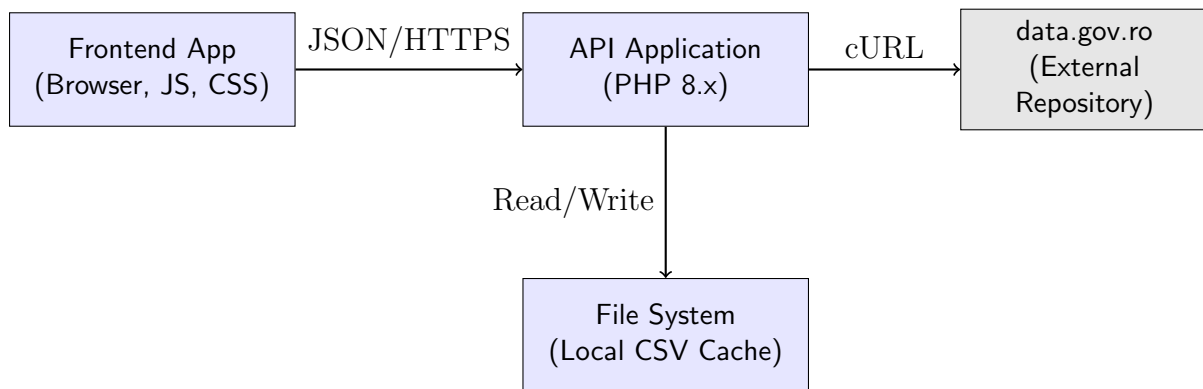
- **Sursa externă indisponibilă:** Dacă `data.gov.ro` nu răspunde la cererea cURL, sistemul API returnează un cod HTTP 502 (Bad Gateway). Frontend-ul captează eroarea și afișează un mesaj de avertizare utilizatorului: „Nu s-au putut încărca datele. Verificați sursa externă.”
- **Date corupte în cache:** Dacă un fișier CSV descărcat este invalid, parserul backend va arunca o excepție. Utilizatorul poate forța reîmprospătarea prin ștergerea cache-ului din interfața de administrare/debug a API-ului.
- **Eroare de memorie la export:** Din cauza volumului mare de date, exportul PDF poate eșua în browsere cu memorie limitată. Sistemul folosește compresie JPEG pentru grafice și procesare asincronă pentru a minimiza acest risc.

3 Diagrams (C4 Model)

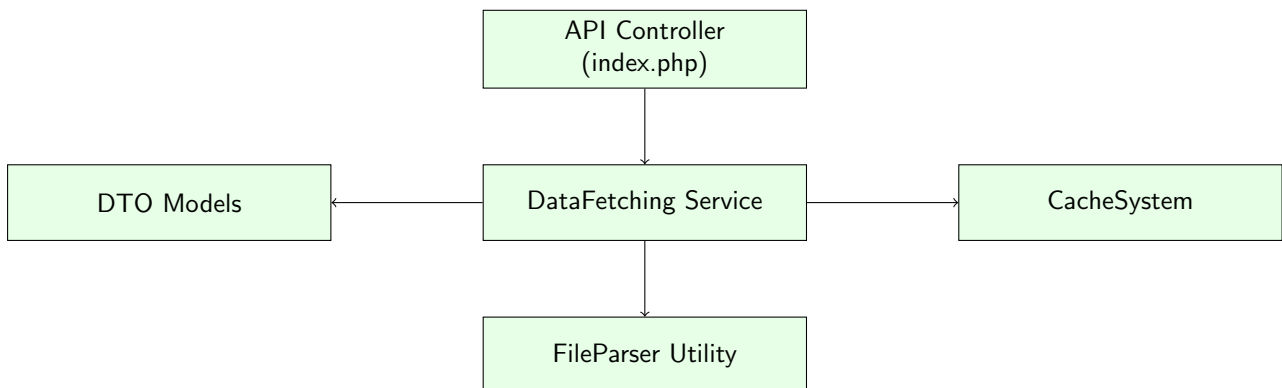
3.1 Level 1: System Context Diagram



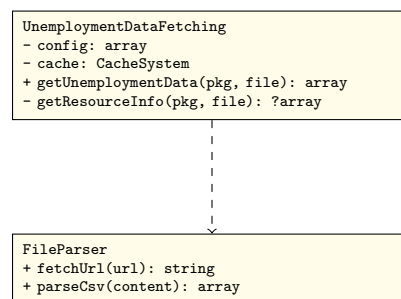
3.2 Level 2: Container Diagram



3.3 Level 3: Component Diagram (Backend API)



3.4 Level 4: Code Diagram (Class Structure Example)



4 Arhitectura Sistemului

Aplicația este construită pe un model **Decoupled Front-to-Back**, utilizând containere Docker pentru a asigura un mediu de rulare izolat, predictibil și ușor de scalat.

4.1 Infrastructură: Dockerization

Sistemul este orchestrat prin `docker-compose` și este compus din două servicii principale:

- **Serviciul API:** Un container PHP-FPM/Apache care rulează nucleul logic de preluare și procesare a datelor. Acesta nu este expus direct utilizatorului final, comunicând doar în rețeaua internă Docker.
- **Serviciul Frontend:** Un container Apache care servește fișierele statice (HTML, CSS, JS) și include un script `api-proxy.php`.

4.2 Comunicarea și Securitatea: API Proxy

Pentru a evita problemele de **CORS (Cross-Origin Resource Sharing)** și pentru a menține API-ul într-o zonă securizată, comunicarea se face printr-un proxy:

1. Frontend-ul (JS) trimite cereri către `api-proxy.php` aflat pe același host.
2. Proxy-ul PHP utilizează cURL pentru a comunica cu containerul `unemployment-api` prin DNS-ul intern al Docker.
3. Acest model permite ascunderea complexității infrastructurii și oferă un punct unic de control pentru cererile către datele externe.

4.3 Separarea Responsabilităților (Separation of Concerns)

- **Backend (Stateless):** Se ocupă strict de fetching, parsare CSV, normalizare (curățarea diacriticelor corupte) și caching. Nu menține stări de sesiune ale utilizatorului.
- **Frontend (Stateful):** Gestionează starea aplicației (`state` object), preferințele de filtrare ale utilizatorului și logica de randare a componentelor vizuale.

5 Backend Documentation

5.1 API Reference

Sistemul Backend oferă o interfață RESTful pentru consumul datelor statistice și gestionarea stării interne (cache).

5.1.1 Endpoint: Fetch Data

- **Metodă:** GET
- **URL:** `/api/{packageName}/{fileName}`
- **Parametri:**

- `packageName`: ID-ul perioadei (ex: `mai2025`, `aprilie2024`).
- `fileName`: Tipul setului de date (`rata.csv`, `medii.csv`, `varste.csv`, `nivel-educatie.csv`).
- **Răspuns**: JSON Array conținând obiectele DTO corespunzătoare setului cerut.

5.1.2 Endpoint: List Cache

- **Metodă**: GET
- **URL**: `/api/cache`
- **Răspuns**: Listă de obiecte `CsvCacheEntry` cu detalii despre mărime și data creării.

5.1.3 Endpoint: Delete Cache

- **Metodă**: DELETE
- **URL**: `/api/cache` (Toate fișierele) sau `/api/cache/{filename}` (Un singur fișier).
- **Răspuns**: `{"success": true, "msg": "..."}`

5.2 DTOs Structure (Models)

Toate modelele implementează `JsonSerializable` pentru o conversie facilă în JSON.

5.2.1 UnemploymentDataBasic

Mapat pe `rata.csv`.

- `string $county`: Numele județului.
- `int $nrUnemployed`: Număr total șomeri.
- `int $nrFemaleUnemployed`: Număr femei șomere.
- `int $nrMaleUnemployed`: Număr bărbați șomeri.
- `int $nrCompensatedUnemployed`: Șomeri indemnizați.
- `int $nrNonCompensatedUnemployed`: Șomeri neindemnizați.
- `float $unemploymentRate`: Rata șomajului (%).

5.2.2 UnemploymentDataPerAgeRange

Mapat pe `varste.csv`.

- `string $county`: Județ.
- `int $under25, $from25to29, $from30to39, $from40to49, $from50to59, $over50`: Distribuție numerică pe vârste.

5.2.3 UnemploymentDataPerEducationLevel

Mapat pe nivel-educatie.csv.

- `string $county`: Județ.
- `int $noStudy, $primaryStudy, $middleStudy, $highStudy, $postHighStudy, $professionalStudy, $universityStudy`: Distribuție după nivelul de școlarizare.

5.2.4 UnemploymentDataPerMedium

Mapat pe medii.csv.

- `int $totalUnemployedUrban, $totalUnemployedRural`: Distribuție Urban vs Rural.

5.3 Caching and Transformation Strategy

Sistemul de caching este implementat în clasa `CacheSystem` și are rolul de a optimiza performanța și de a asigura stabilitatea aplicației în cazul în care sursa externă este indisponibilă temporar.

5.3.1 Mecanismul de Stocare

Datele sunt stocate local în directorul `API/cache/` sub formă de fișiere CSV brute. Denumirea fișierelor urmează schema `{package_name}_{file_name}`, asigurând unicitatea resurselor pentru fiecare lună și criteriu.

5.3.2 Ciclul de Viață al Datelor (TTL)

Aplicația implementează o politică de expirare automată a datelor:

- **Durata de viață**: Fișierele sunt considerate valide timp de 7 zile (`CACHE_LIFETIME = 604800` secunde).
- **Curățare Automată**: La fiecare cerere către `index.php`, sistemul scanează directorul de cache și șterge fișierele a căror dată de modificare (`filemtime`) este mai veche decât pragul setat.
- **Validare la Fetch**: În `UnemploymentDataFetching`, înainte de a iniția o cerere cURL, se verifică existența fișierului în cache. Dacă acesta lipsește, se descarcă de pe `data.gov.ro` și se salvează local prin metoda `CacheSystem::put()`.

5.3.3 De ce convertim din CSV în JSON?

1. **Curățarea Datelor**: Fișierele sursă conțin adesea erori de encoding (ex: „Caraș-Severin” scris greșit). API-ul corectează aceste erori în timpul conversiei.
2. **Performanță**: JSON-ul este mult mai rapid de parsat în Frontend decât un CSV brut, reducând timpul de blocare a thread-ului principal în browser.
3. **Tipizarea și Normalizarea**: Conversia permite transformarea string-urilor din CSV în tipuri numerice (`int/float`) și eliminarea punctelor de separare a mii, asigurând calcule matematice corecte în grafice.

6 Frontend Implementation

6.1 Mecanismul de Export

Aplicația suportă cinci formate de export, fiecare folosind tehnici diferite de procesare a datelor.

6.1.1 Export CSV (Date Tabelare)

Sistemul generează fișiere CSV direct în browser:

- **Separator:** Se folosește punct și virgulă (;) pentru compatibilitate cu setările regionale europene ale Microsoft Excel.
- **Diacritice:** Se prefixează conținutul cu un **Byte Order Mark (BOM)** (\uFEFF) pentru a forța Excel să recunoască codarea UTF-8 și să afișeze corect caracterele românești.

6.1.2 Export SVG (Vectorial)

Deoarece Chart.js utilizează elementul `<canvas>` (bazat pe pixeli), exportul vectorial este realizat prin încapsulare:

1. Se capturează canvas-ul curent ca un **DataURL (PNG)**.
2. Se construiește un fișier XML SVG care conține un tag `<image>` cu sursa setată la DataURL-ul capturat.
3. Această metodă permite integrarea graficului în documente vectoriale menținând rezoluția originală de randare.

6.1.3 Export PDF (Raport Complet)

Folosește **jsPDF** și **jspdf-autotable** pentru un document multi-pagină:

- **Fonturi Custom:** Embedează fontul **Roboto** (Normal/Bold) sub formă de Base64 pentru suportul diacriticelor.
- **Integritate:** Spre deosebire de alte metode, PDF-ul re-parcurește întreg setul de date din `state.data` pentru a genera un tabel complet, ignorând limitările de paginare ale UI-ului.
- **Print Workflow:** Utilizează `pdf.autoPrint()` pentru a declanșa automat dialogul de sistem, facilitând utilizarea virtualizării precum `cups-pdf`.

6.1.4 Export SQL (Migrare și Interoperabilitate)

Aplicația permite transformarea datelor vizualizate în scripturi SQL gata de importat:

- **Generare Dinamică:** Sistemul construiește instrucțiuni `CREATE TABLE` și `INSERT INTO` bazate pe structura DTO-ului curent.
- **Sanitizare:** Datele sunt curățate pentru a preveni erorile de sintaxă SQL (ex: escape pentru ghilimelele din numele județelor).

- **Cazul de utilizare:** Facilitează integrarea rapidă a datelor de la **data.gov.ro** în sisteme de management al bazelor de date (RDBMS) externe pentru analize complexe.

6.1.5 Export JSON (Sursă Brută)

Pentru dezvoltatori și analiști, aplicația oferă exportul stării curente în format JSON:

- **Structură Fidelă:** Fișierul generat păstrează ierarhia obiectelor JavaScript din `state`, oferind acces la toate metadatele procesate de API.
- **Portabilitate:** Formatul JSON asigură cea mai mare compatibilitate cu alte limbaje de programare (Python, R, Java) pentru procesări ulterioare în fluxuri de Data Science.

6.2 Harta Interactivă (Leaflet)

Afișarea hărții se bazează pe biblioteca **Leaflet**.

1. **GeoJSON:** Granitele județelor sunt încărcate dintr-un fișier extern GeoJSON.
2. **Colorare:** Funcția `getColor(rate)` mapează rata șomajului pe o paletă de culori de la verde la roșu.
3. **Normalizare:** Deoarece numele județelor în GeoJSON diferă de cele din sursa oficială, se folosește o funcție de normalizare a string-urilor.

6.3 Randarea Graficelor (Chart.js)

Graficele sunt desenate dinamic în elemente `<canvas>`.

- **Bar Chart:** Folosit pentru evoluția pe județe. Suportă modul „Stacked” pentru criterii multiple și modul „Comparison” pentru două perioade.
- **Donut Chart:** Folosit pentru distribuția procentuală. Centrul graficului afișează automat procentul celei mai mari categorii.